

Post-contenido — Manejo del DEBUG

Exploración, Ensamblado y Ejecución Paso a Paso en DOSBox

Arquitectura de Computadores
Unidad 3

Juan Diego Rojas Rey
Código: 1152512

Universidad Francisco de Paula Santander

Descripción general

Este informe documenta el desarrollo del laboratorio guiado sobre el depurador DEBUG de DOS, ejecutado en un entorno DOSBox. El laboratorio consta de dos partes. En la Parte 1 se exploran el estado inicial del procesador, los comandos de inspección de memoria (R, F, D, U) y la modificación puntual de memoria con E, verificada mediante direccionamiento directo a memoria. En la Parte 2 se ensamblan programas propios con el comando A, se ejecutan instrucción a instrucción con T registrando su evolución en tablas de traza, y se analiza el mecanismo de bucle LOOP frente a una implementación equivalente construida con DEC y JNZ, además del comando G como alternativa de verificación rápida frente a T.

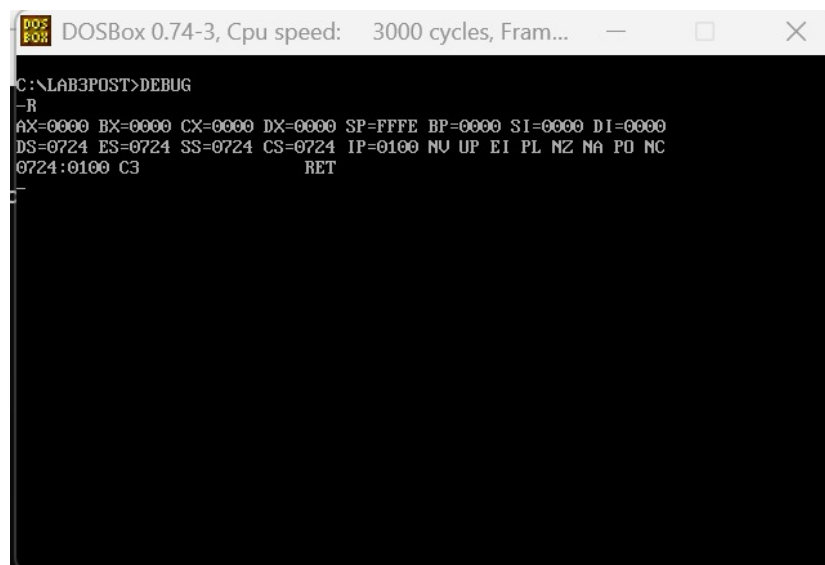
Todos los comandos se ejecutaron sobre el segmento 0724, valor asignado por DOSBox al iniciar la sesión de DEBUG, en lugar del segmento 1357 usado como ejemplo en la guía. Esta diferencia es esperada, ya que el segmento del PSP depende de la memoria disponible en cada sesión, y no afecta ninguno de los resultados ni de las verificaciones solicitadas.

Parte 1: Exploración con DEBUG en DOSBox

Checkpoint 1 — Estado de registros

Se ejecutó el comando R sin argumentos al iniciar DEBUG. Los cuatro registros de propósito general (AX, BX, CX, DX) se observaron en cero, SP en FFFE (tope inicial de la pila) y los cuatro registros de segmento (DS, ES, SS, CS) apuntando al mismo valor de segmento, 0724, correspondiente al párrafo de memoria asignado por el DOS al PSP del programa. El IP se ubicó en 0100, la primera dirección ejecutable después del PSP.

Una diferencia respecto al ejemplo de la guía: la instrucción mostrada en 0724:0100 fue C3 (RET), mientras que la guía ilustra CD20 (INT 20). Ambos son bytes residuales de memoria no inicializada que el DOS deja en esa posición al momento de cargar DEBUG, así que el byte específico varía entre sesiones y no representa un error.



```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
C:\LAB3POST>DEBUG
-R
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0100 NU UP EI PL NZ NA PO NC
0724:0100 C3          RET
```

Figura 1. Checkpoint 1 — Estado inicial de registros (comando R).

Checkpoint 2 — Volcado hexadecimal anotado

Se rellenaron 64 bytes (0x40) a partir de DS:0200 con el comando F 200 L40 AB CD EF, y se volcó la región con D 200 L40. El patrón AB CD EF se repite cíclicamente a lo largo de las cuatro filas del volcado, confirmando que F distribuye el patrón indicado de forma continua hasta completar el rango.

La salida del comando D se organiza en tres columnas: a la izquierda, la dirección de segmento y desplazamiento del primer byte de cada fila; en el centro, dieciséis bytes en hexadecimal separados por un guion a la mitad para facilitar la lectura; y a la derecha, la representación ASCII de esos mismos bytes, donde cada carácter no imprimible se sustituye por un punto. En este caso todos los bytes son puntos, porque 0xAB, 0xCD y 0xEF están fuera del rango imprimible (0x20–0x7E).

```
0724:0100  F 200 L40 AB CD EF
0724:0100  D 200 L40
0724:0200  AB CD EF AB CD EF AB CD EF AB CD EF AB CD EF AB .....
0724:0210  CD EF AB CD EF AB CD EF AB CD EF AB CD EF AB CD .....
0724:0220  EF AB CD EF AB CD EF AB CD EF AB CD EF AB CD EF .....
0724:0230  AB CD EF AB CD EF AB CD EF AB CD EF AB CD EF AB .....
```

Figura 2. Checkpoint 2 — Volcado del patrón AB CD EF (comando D).

Checkpoint 3 — Sesión de ensamblado y desensamblado

Se ensambló con A 100 el programa MOV AX,0005 / MOV BX,0003 / ADD AX,BX / INT 20, y se verificó con U 100 109:

Dirección	Bytes	Instrucción
0724:0100	B8 05 00	MOV AX,0005
0724:0103	BB 03 00	MOV BX,0003
0724:0106	01 D8	ADD AX,BX
0724:0108	CD 20	INT 20

Las tres primeras instrucciones y la última coinciden exactamente con el ejemplo de la guía. La única diferencia está en ADD AX,BX, que la guía codifica como 03 C3 y que en esta sesión se codificó como 01 D8.

Nota sobre la codificación alterna de ADD AX,BX

En el conjunto de instrucciones x86 existen dos codificaciones válidas para ADD AX,BX, ambas de 2 bytes y con el mismo efecto: 03 C3 usa el opcode 03 (ADD registro, registro/memoria), donde el campo reg del byte ModRM lleva el destino y r/m la fuente; 01 D8 usa el opcode 01 (ADD registro/memoria, registro), donde es al revés. Separando el byte ModRM en sus tres campos (mod, reg, r/m), con AX = 000 y BX = 011: C3 = 11 000 011, con reg = 000 (AX) y r/m = 011 (BX), de modo que con el opcode 03 el resultado es AX = AX + BX; D8 = 11 011 000, con reg = 011 (BX) y r/m = 000 (AX), de modo que con el opcode 01 el resultado es igualmente AX = AX + BX. El resultado es idéntico y ambas codificaciones ocupan 2 bytes, por lo que el programa completo mide 10 bytes en cualquiera de los dos casos. Qué codificación elige el ensamblador de DEBUG para una instrucción con dos operandos registro del mismo tamaño es una decisión interna del programa y no afecta la semántica ni el tamaño del código. Esta misma equivalencia se repite en varias instrucciones ADD de la Parte 2 del laboratorio.

```

-D 0 L20
0724:0000 CD 20 FF 9F 00 EA FF FF-AD DE 7E 1D 92 01 00 00 . .....~.....
0724:0010 18 01 10 01 18 01 92 01-01 01 01 00 02 FF FF FF .....
-U 100 L10
0724:0100 C3 RET
0724:0101 0000 ADD [BX+SI],AL
0724:0103 0000 ADD [BX+SI],AL
0724:0105 0000 ADD [BX+SI],AL
0724:0107 0000 ADD [BX+SI],AL
0724:0109 0000 ADD [BX+SI],AL
0724:010B 0000 ADD [BX+SI],AL
0724:010D 0000 ADD [BX+SI],AL
0724:010F 0000 ADD [BX+SI],AL
-A 100
0724:0100 MOV AX,0005
0724:0103 MOV BX,0003
0724:0106 ADD AX,BX
0724:0108 INT 20
0724:010A
-U 100 109
0724:0100 B80500 MOV AX,0005
0724:0103 BB0300 MOV BX,0003
0724:0106 01D8 ADD AX,BX
0724:0108 CD20 INT 20

```

Figura 3. Checkpoint 3 — Ensamblado (A) y desensamblado (U).

Checkpoint 4 — Modificación de memoria y direccionamiento directo

Se limpió una región de 16 bytes en DS:0300 con F 300 L10 00, y se verificó con D 300 L10 que los 16 bytes quedaron en cero. Luego se escribieron dos bytes puntuales con E 300 78 56, y un segundo D 300 L10 confirmó que solo las dos primeras posiciones cambiaron, a 78 56, mientras los 14 bytes restantes permanecieron en 00. Leídos como palabra little-endian, estos dos bytes representan el valor 0x5678.

A continuación se ensambló en 0724:0320 (una dirección distinta a la del programa del Checkpoint 3, para no sobrescribirlo) la instrucción MOV AX,[0300] seguida de INT 20, y se verificó con U 320 324:

Dirección	Bytes	Instrucción
0724:0320	A1 00 03	MOV AX,[0300]
0724:0323	CD 20	INT 20

Se ejecutó la instrucción con T (IP en 0320), y el registro AX pasó a valer 5678, confirmando que el direccionamiento directo a memoria leyó correctamente el valor escrito previamente con E.

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
0724:0108 CD20          INT     20
-F 300 L10 00
-D 300 L10
0724:0300 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
-E 300 78 56
-D 300 L10
0724:0300 78 56 00 00 00 00 00 00-00 00 00 00 00 00 00 xU.....
-A 320
0724:0320 MOV AX,[0300]
0724:0323 INT 20
0724:0325
-U 320 324
0724:0320 A10003      MOV     AX,[0300]
0724:0323 CD20          INT     20
-T =320
AX=5678 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0323 NV UP EI PL NZ NA PO NC
0724:0323 CD20          INT     20
-R IP
IP 0323 :0320
-T
AX=5678 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0323 NV UP EI PL NZ NA PO NC
0724:0323 CD20          INT     20

```

Figura 4. Checkpoint 4 — Secuencia F, D, E, D, A, U y T.

Decisión técnica — Verificación no destructiva de una escritura en memoria (Paso 12)

Para confirmar que la escritura del Paso 11 tuvo el efecto esperado, se eligió el comando D, porque es el único de los cuatro candidatos que únicamente lee la memoria y no modifica ningún dato. Invocar E 300 sin la lista de bytes no es una opción segura, ya que el DEBUG entra en modo interactivo, muestra el valor actual de cada byte y espera una tecla, de modo que una pulsación equivocada sobrescribiría justamente la memoria que se pretendía verificar. En cambio, con la lista completa de bytes, como en el Paso 11, la escritura es determinista y no queda esperando entrada. El comando F tampoco es adecuado: aunque opera sobre un rango como D, su función es rellenar, por lo que reescribiría todo el rango con el patrón indicado y destruiría la evidencia que se busca comprobar. El comando R, además de mostrar registros, los modifica cuando recibe el nombre de un registro como argumento, y en todo caso no muestra el contenido de la memoria. D solo copia los bytes a la pantalla sin alterar registros ni memoria, y esa propiedad de solo lectura es exactamente la que se necesita para comprobar sin riesgo que las direcciones 0300 y 0301 contienen 78 56 y que los otros catorce bytes de la región siguen en cero.

Decisión técnica — Direccionamiento inmediato frente a directo a memoria (Paso 14)

Aunque MOV AX,0005 (B8 05 00) y MOV AX,[0300] (A1 00 03) ocupan tres bytes cada una, el direccionamiento directo a memoria requiere un ciclo adicional de acceso al bus. En el modo inmediato, el valor 0005 viaja dentro del propio flujo de instrucciones, por lo que el procesador lo obtiene durante la misma extracción de la instrucción, sin acceso adicional. En el modo directo, los dos últimos bytes de la instrucción contienen una dirección, no el dato en sí, de modo que el procesador debe resolver esa dirección y realizar un segundo acceso a memoria para leer el valor que se cargará en AX. El estudiante preferiría el modo directo a memoria cuando el valor puede cambiar en tiempo de ejecución, como ocurre con el 5678 escrito previamente con E en el Paso 11, porque una constante inmediata queda fija en el momento del ensamblado y no reflejaría ese tipo de cambios; el modo inmediato conviene, en cambio, cuando el valor es una constante conocida de antemano y se privilegia la velocidad. Con el comando U, sin

ejecutar ninguna instrucción, se distingue cada codificación con solo observar el opcode y el operando mostrado: el opcode B8 acompañado del valor 0005 indica direccionamiento inmediato, mientras que el opcode A1 acompañado del operando entre corchetes [0300] indica direccionamiento directo a memoria.

Parte 2: Ensamblado y Ejecución Paso a Paso

Checkpoint 1 — Tabla de traza: programa de suma

Se ensambló en 0724:0100 el programa MOV AX,000A / MOV BX,0005 / MOV CX,0003 / ADD AX,BX / ADD AX,CX / INT 20, verificado con U 100 10E. Al igual que en la Parte 1, ambas instrucciones ADD se codificaron con el opcode alternativo: ADD AX,BX como 01 D8 y ADD AX,CX como 01 C8, en lugar de 03 C3 y 03 C1 como en la guía. Se aplica la misma equivalencia explicada en el Checkpoint 3 de la Parte 1.

Instrucción	AX	BX	CX	IP sig.	ZF	CF	SF
MOV AX,000A	000A	0000	0000	0103	0	0	0
MOV BX,0005	000A	0005	0000	0106	0	0	0
MOV CX,0003	000A	0005	0003	0109	0	0	0
ADD AX,BX	000F	0005	0003	010B	0	0	0
ADD AX,CX	0012	0005	0003	010D	0	0	0

Al finalizar las instrucciones ADD, AX vale 0012 (18 decimal), el resultado esperado de $10 + 5 + 3$. En la última instrucción (ADD AX,CX), además, se activaron las banderas AC (acarreo auxiliar) y PE (paridad par), visibles en la captura, porque la suma $0x0F + 0x03$ genera acarreo desde el nibble bajo; esto no afecta a ZF, CF ni SF, que permanecen en cero en las cinco filas.

```
DOS BOX DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
C:\LAB3POST\LAB3POST>DEBUG
-A 100
0724:0100 MOV AX,000A
0724:0103 MOV BX,0005
0724:0106 MOV CX,0003
0724:0109 ADD AX,BX
0724:010B ADD AX,CX
0724:010D INT 20
0724:010F
-T
AX=000A BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0103 NU UP EI PL NZ NA PO NC
0724:0103 BB0500 MOV BX,0005
-T
AX=000A BX=0005 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0106 NU UP EI PL NZ NA PO NC
0724:0106 B90300 MOV CX,0003
-T
AX=000A BX=0005 CX=0003 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0109 NU UP EI PL NZ NA PO NC
0724:0109 01D8 ADD AX,BX
-T
AX=000A BX=0005 CX=0003 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0103 NU UP EI PL NZ NA PO NC
0724:0103 BB0500 MOV BX,0005
-T
AX=000A BX=0005 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0106 NU UP EI PL NZ NA PO NC
0724:0106 B90300 MOV CX,0003
-T
AX=000A BX=0005 CX=0003 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0109 NU UP EI PL NZ NA PO NC
0724:0109 01D8 ADD AX,BX
-T
AX=000F BX=0005 CX=0003 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=010B NU UP EI PL NZ NA PE NC
0724:010B 01C8 ADD AX,CX
-T
AX=0012 BX=0005 CX=0003 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=010D NU UP EI PL NZ AC PE NC
0724:010D CD20 INT 20
```

Figura 5. Checkpoint 1 — Traza del programa de suma (comando T).

Checkpoint 2 — Tabla de traza: programa con bucle LOOP

Se ensambló en 0724:0100 el programa MOV AX,0000 / MOV CX,0004 / ADD AX,+02 / LOOP 0106 / INT 20, verificado con U 100 10D:

Dirección	Bytes	Instrucción
0724:0100	B8 00 00	MOV AX,0000
0724:0103	B9 04 00	MOV CX,0004
0724:0106	83 C0 02	ADD AX,+02
0724:0109	E2 FB	LOOP 0106
0724:010B	CD 20	INT 20

Se observan dos diferencias respecto al ejemplo de la guía, ambas sin efecto sobre el resultado del programa. Primero, ADD AX,0002 se codificó como 83 C0 02 (instrucción con inmediato de 8 bits extendido con signo, que DEBUG muestra como ADD AX,+02) en lugar de 05 02 00 como en la guía; ambas formas ocupan 3 bytes y producen el mismo resultado. Segundo, como consecuencia de lo anterior, la instrucción LOOP quedó ubicada en 0109 en lugar de 010A; el desplazamiento relativo se recalculó correctamente como $0x0106 - 0x010B = -5 = 0xFB$, el mismo valor que muestra la guía, ya que ambas direcciones (destino y origen del salto) se desplazaron igual.

Iter.	Instrucción	AX desp.	CX desp.	IP sig.	¿LOOP salta?
Inicio	MOV AX,0000	0000	0000	0103	No aplica
Inicio	MOV CX,0004	0000	0004	0106	No aplica
1	ADD AX,+02	0002	0004	0109	No aplica
1	LOOP 0106	0002	0003	0106	Sí
2	ADD AX,+02	0004	0003	0109	No aplica
2	LOOP 0106	0004	0002	0106	Sí
3	ADD AX,+02	0006	0002	0109	No aplica
3	LOOP 0106	0006	0001	0106	Sí
4	ADD AX,+02	0008	0001	0109	No aplica
4	LOOP 0106	0008	0000	010B	No
Fin	INT 20	0008	0000	Terminó	No aplica

AX finaliza en 0008 (8 decimal), el resultado esperado de sumar 2 cuatro veces. LOOP no modifica ZF, CF ni SF; estas banderas permanecen en 0 (NZ, NC) durante toda la traza, y solo la bandera de paridad alterna entre PO y PE según el número de bits en 1 del resultado, sin relevancia para el control del bucle.

```

DOS
Box
DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
C:\LAB3POST\LAB3POST>DEBUG
-A 100
0724:0100 MDU AX,0000
0724:0103 MDU CX,0004
0724:0106 ADD AX,0002
0724:0109 LOOP 0106
0724:010B INT 20
0724:010D
-U 100 10D
0724:0100 B00000 MDU AX,0000
0724:0103 B90400 MDU CX,0004
0724:0106 83C002 ADD AX,+02
0724:0109 E2FB LOOPW 0106
0724:010B CD20 INT 20
0724:010D CD20 INT 20
-T
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0103 NU UP EI PL NZ NA PO NC
0724:0103 B90400 MDU CX,0004
-T
AX=0000 BX=0000 CX=0004 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0106 NU UP EI PL NZ NA PO NC
0724:0106 83C002 ADD AX,+02
-----
-T
AX=0002 BX=0000 CX=0004 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0109 NU UP EI PL NZ NA PO NC
0724:0109 E2FB LOOPW 0106
-T
AX=0002 BX=0000 CX=0003 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0106 NU UP EI PL NZ NA PO NC
0724:0106 83C002 ADD AX,+02
-T
AX=0004 BX=0000 CX=0003 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0109 NU UP EI PL NZ NA PO NC
0724:0109 E2FB LOOPW 0106
-T
AX=0004 BX=0000 CX=0002 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0106 NU UP EI PL NZ NA PO NC
0724:0106 83C002 ADD AX,+02
-T
AX=0006 BX=0000 CX=0002 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0109 NU UP EI PL NZ NA PE NC
0724:0109 E2FB LOOPW 0106
C-
-T
AX=0004 BX=0000 CX=0002 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0106 NU UP EI PL NZ NA PO NC
0724:0106 83C002 ADD AX,+02
-T
AX=0006 BX=0000 CX=0002 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0109 NU UP EI PL NZ NA PE NC
0724:0109 E2FB LOOPW 0106
-T
AX=0006 BX=0000 CX=0001 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0106 NU UP EI PL NZ NA PE NC
0724:0106 83C002 ADD AX,+02
-T
AX=0008 BX=0000 CX=0001 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0109 NU UP EI PL NZ NA PO NC
0724:0109 E2FB LOOPW 0106
-T
AX=0008 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=010B NU UP EI PL NZ NA PO NC
0724:010B CD20 INT 20
-T
Program terminated normally (0000)

```

Figura 6. Checkpoint 2 — Traza del bucle LOOP (comando T).

Checkpoint 3 — Análisis del código máquina

El volcado del programa con bucle LOOP (D 100 L0D) mostró la secuencia B8 00 00 B9 04 00 83 C0 02 E2 FB CD 20.

Bytes	Instrucción	Tamaño
B8 00 00	MOV AX,0000	3 bytes
B9 04 00	MOV CX,0004	3 bytes
83 C0 02	ADD AX,+02	3 bytes
E2 FB	LOOP 0106	2 bytes
CD 20	INT 20	2 bytes

El total es $3 + 3 + 3 + 2 + 2 = 13$ bytes. Se corrige aquí una inconsistencia de la guía, que indica en el enunciado del Paso 8 que el programa ocupa 12 bytes, pero cuya propia suma de los tamaños de instrucción ($3+3+2+2+2$, en su codificación de referencia) da 12; en la ejecución real documentada en este informe, la suma de los tamaños reales de instrucción da 13 bytes, que es también la cifra que la guía menciona en su párrafo de cierre del Paso 8 ("un total de 13 bytes"). Se toma 13 bytes como el valor correcto, consistente con el volcado hexadecimal obtenido.

Checkpoint 3 — Tabla de traza y comparación: bucle equivalente con DEC/JNZ

Se ensambló en 0724:0200 (dirección distinta a la del bucle con LOOP, para no sobrescribirlo) el programa MOV AX,0000 / MOV CX,0004 / ADD AX,0002 / DEC CX / JNZ 0206 / INT 20, verificado con U 200 20C:

Dirección	Bytes	Instrucción
0724:0200	B8 00 00	MOV AX,0000
0724:0203	B9 04 00	MOV CX,0004
0724:0206	83 C0 02	ADD AX,+02
0724:0209	49	DEC CX
0724:020A	75 FA	JNZ 0206
0724:020C	CD 20	INT 20

DEC CX ocupa 1 byte y JNZ 0206 ocupa 2 bytes (75 FA), con desplazamiento $0x0206 - 0x020C = -6 = 0xFA$. Frente al único LOOP 0106 de 2 bytes del Checkpoint 2, aquí se necesitan 3 bytes en dos instrucciones separadas para lograr el mismo efecto observable.

Iter.	Instrucción	AX desp.	CX desp.	IP sig.	ZF	¿JNZ salta?
Inicio	MOV AX,0000	0000	0000	0203	0	No aplica
Inicio	MOV CX,0004	0000	0004	0206	0	No aplica
1	ADD AX,+02	0002	0004	0209	0	No aplica
1	DEC CX	0002	0003	020A	0	No aplica

Iter.	Instrucción	AX desp.	CX desp.	IP sig.	ZF	¿JNZ salta?
1	JNZ 0206	0002	0003	0206	0	Sí
2	ADD AX,+02	0004	0003	0209	0	No aplica
2	DEC CX	0004	0002	020A	0	No aplica
2	JNZ 0206	0004	0002	0206	0	Sí
3	ADD AX,+02	0006	0002	0209	0	No aplica
3	DEC CX	0006	0001	020A	0	No aplica
3	JNZ 0206	0006	0001	0206	0	Sí
4	ADD AX,+02	0008	0001	0209	0	No aplica
4	DEC CX	0008	0000	020A	1 (ZR)	No aplica
4	JNZ 0206	0008	0000	020C	1 (ZR)	No
Fin	INT 20	0008	0000	Terminó	1	No aplica

AX finaliza en 0008, el mismo resultado que con LOOP, confirmando que ambos mecanismos son semánticamente equivalentes. La bandera ZF permanece en 0 mientras DEC CX no llega a cero, y solo pasa a 1 (ZR) en la última iteración, cuando CX llega a 0000; es ese cambio de ZF el que hace que JNZ no salte en la cuarta iteración.

Comparación de instrucciones ejecutadas. La traza del bucle con LOOP (Checkpoint 2) requirió 11 invocaciones de T (2 de inicialización + 4 × (ADD + LOOP) + 1 de terminación), mientras que la traza con DEC/JNZ requirió 15 (2 de inicialización + 4 × (ADD + DEC + JNZ) + 1 de terminación), es decir, 4 instrucciones adicionales, exactamente una por cada una de las 4 iteraciones, correspondientes a la instrucción de control extra que DEC/JNZ necesita frente a LOOP.

Como demostración del comando G, se restableció IP a 0200 y se ejecutó G 20C, obteniendo directamente AX=0008 sin pasar por ninguno de los pasos intermedios. Con R AX se confirmó el valor final del registro (0008) sin haber observado ningún estado intermedio del programa.

```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram... — □ ×
-T
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0203 NU UP EI PL NZ NA PO NC
0724:0203 B90400      MOV     CX,0004
-T
AX=0000 BX=0000 CX=0004 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0206 NU UP EI PL NZ NA PO NC
0724:0206 83C002      ADD     AX,+02
-T
AX=0002 BX=0000 CX=0004 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0209 NU UP EI PL NZ NA PO NC
0724:0209 49          DEC     CX
-T
AX=0002 BX=0000 CX=0003 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=020A NU UP EI PL NZ NA PE NC
0724:020A 75FA          JNZ     0206
-T
AX=0002 BX=0000 CX=0003 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0206 NU UP EI PL NZ NA PE NC
0724:0206 83C002      ADD     AX,+02
-T
AX=0004 BX=0000 CX=0003 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0209 NU UP EI PL NZ NA PO NC
0724:0209 49          DEC     CX
DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram... — □ ×
-T
AX=0004 BX=0000 CX=0002 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=020A NU UP EI PL NZ NA PO NC
0724:020A 75FA          JNZ     0206
-T
AX=0004 BX=0000 CX=0002 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0206 NU UP EI PL NZ NA PO NC
0724:0206 83C002      ADD     AX,+02
-T
AX=0006 BX=0000 CX=0002 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0209 NU UP EI PL NZ NA PE NC
0724:0209 49          DEC     CX
-T
AX=0006 BX=0000 CX=0001 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=020A NU UP EI PL NZ NA PO NC
0724:020A 75FA          JNZ     0206
-T
AX=0006 BX=0000 CX=0001 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0206 NU UP EI PL NZ NA PO NC
0724:0206 83C002      ADD     AX,+02
-T
AX=0008 BX=0000 CX=0001 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0209 NU UP EI PL NZ NA PO NC
0724:0209 49          DEC     CX
DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram... — □ ×
0724:0209 49          DEC     CX
-T
AX=0006 BX=0000 CX=0001 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=020A NU UP EI PL NZ NA PO NC
0724:020A 75FA          JNZ     0206
-T
AX=0006 BX=0000 CX=0001 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0206 NU UP EI PL NZ NA PO NC
0724:0206 83C002      ADD     AX,+02
-T
AX=0008 BX=0000 CX=0001 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=0209 NU UP EI PL NZ NA PO NC
0724:0209 49          DEC     CX
-T
AX=0008 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=020A NU UP EI PL ZR NA PE NC
0724:020A 75FA          JNZ     0206
-T
AX=0008 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=020C NU UP EI PL ZR NA PE NC
0724:020C CD20      INT     20
-T
Program terminated normally (0000)
-R IP
IP 0100 :0200
-G 20C
AX=0008 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=0724 ES=0724 SS=0724 CS=0724 IP=020C NU UP EI PL ZR NA PE NC
0724:020C CD20      INT     20
-R AX
AX 0008 :
```

Figura 7. Checkpoint 3 — Traza del bucle DEC/JNZ y demostración de G.

Decisión técnica — Selección de mecanismo de control de bucle: LOOP vs. DEC/JNZ (Paso 10)

Para un bucle contador simple como este, en el que CX no se necesita para ningún otro propósito dentro del cuerpo del bucle, el estudiante recomienda usar LOOP, porque su codificación ocupa 2 bytes de código máquina (E2 FB) en una sola instrucción, frente a los 3 bytes que ocupan DEC CX y JNZ 0206 juntos (49 75 FA) en dos instrucciones separadas. Esta diferencia se traduce directamente en que el procesador debe extraer (fetch) una instrucción por iteración con LOOP, y dos instrucciones por iteración con DEC/JNZ, lo cual implica más accesos a memoria de código y, en consecuencia, mayor tiempo de ejecución acumulado a medida que crece el número de iteraciones, como se observó en este laboratorio con 11 pasos frente a 15 para el mismo resultado. Sin embargo, el estudiante preferiría DEC/JNZ sobre LOOP en dos escenarios concretos: primero, si el cuerpo del bucle necesitara reutilizar CX para otra operación aritmética distinta al conteo, ya que LOOP exige que CX se dedique exclusivamente a contar iteraciones; segundo, si la condición de salida del bucle no fuera simplemente "CX $\neq$ 0" sino el resultado de una comparación evaluada con CMP, caso en el cual el salto condicional adecuado (por ejemplo JE, JG o JL) reemplazaría a JNZ y LOOP dejaría de ser aplicable. Para verificar con U, sin ejecutar ninguna instrucción, cuál de las dos versiones ocupa menos bytes, basta con desensamblar ambos rangos de memoria y comparar la cantidad de bytes hexadecimal que U muestra junto a cada instrucción, sumándolos por separado para cada versión del bucle.

Decisión técnica — Comando de verificación para bucles de muchas iteraciones: T vs. G (Paso 12)

Para verificar el resultado final de un bucle con 100 iteraciones, G 20C resulta mucho más práctico que repetir T decenas de veces, porque ejecuta el programa a velocidad completa hasta la dirección indicada como punto de interrupción, sin que el usuario deba invocar manualmente un comando por cada instrucción intermedia; lo que se pierde al usar G en lugar de T es precisamente esa visibilidad instrucción por instrucción, es decir, el valor de los registros y de las banderas después de cada paso individual, información que no puede recuperarse una vez el programa ya terminó de ejecutarse hasta el punto de interrupción. En este laboratorio fue necesario usar T, y no G, en los Checkpoints 1, 2 y 3 de la Parte 2 (Pasos 4, 7 y 11 de la guía), porque la tarea en cada uno de ellos consistía en completar una tabla de traza instrucción por instrucción, y esa granularidad —el estado exacto de AX, BX, CX y las banderas después de cada instrucción individual— es algo que únicamente T puede ofrecer; G solo entrega el estado final en la dirección de destino, y habría sido imposible reconstruir con G los valores intermedios que las tablas de traza exigen. Tras ejecutar G 20C, para confirmar únicamente el valor final de AX sin haber observado ningún paso intermedio, el estudiante usaría el comando R AX, que muestra el contenido actual de ese registro específico sin desensamblar código ni volcar memoria adicional.

Conclusiones

El desarrollo de este laboratorio permitió comprobar de forma práctica varios principios fundamentales de la arquitectura x86 en modo real. En primer lugar, quedó demostrado que en el modo real no existe distinción entre código y datos: cualquier byte al que apunte CS:IP es interpretado como instrucción, como se observó al desensamblar bytes en cero como ADD [BX+SI],AL antes de ensamblar el primer programa. En segundo lugar, se confirmó que una misma instrucción en ensamblador puede tener más de una codificación binaria válida y equivalente, como ocurrió repetidamente con las instrucciones ADD de dos registros a lo largo de todo el laboratorio, y como se documentó también con la forma extendida con signo de ADD AX,imm8 frente a ADD AX,imm16. En tercer lugar, se evidenció la diferencia práctica

entre comandos de solo lectura y comandos que modifican el estado del procesador o de la memoria, y por qué esa distinción importa al momento de verificar resultados sin introducir efectos secundarios no deseados. Finalmente, la comparación entre LOOP y DEC/JNZ, y entre T y G, mostró que la elección del comando o del mecanismo correcto depende del objetivo concreto de cada tarea: compacidad de código frente a flexibilidad del registro contador, y observación detallada frente a verificación rápida de un resultado final.